

Standard Parasitic Extraction Format (SPEF)

This appendix describes the Standard Parasitic Extraction Format (SPEF). It is part of the IEEE Std 1481.

C.1 Basics

SPEF allows the description of parasitic information of a design (R, L and C) in an ASCII exchange format. A user can read and check values in a SPEF file, though the user would never create this file manually. It is mainly

used to pass parasitic information from one tool to another. Figure C-1 shows that SPEF can be generated by tools such as a place-and-route tool or a parasitic extraction tool, and then used by a timing analysis tool, in circuit simulation or to perform crosstalk analysis.



Figure C-1 *SPEF is a tool exchange medium.*

Parasitics can be represented at many different levels. SPEF supports the distributed net model, the reduced net model and the lumped capacitance model. In the distributed net model (D_NET), each segment of a net route has its own R and C . In a reduced net model (R_NET), only a single reduced R and C is considered on the load pins of the net and a pie model ($C-R-C$) is considered on the driver pin of the net. In a lumped capacitance model, only a single capacitance is specified for the entire net. Figure C-2 shows an example of a physical net route. Figure C-3 shows the distributed net mod-

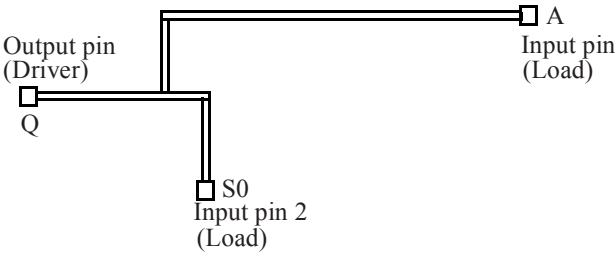


Figure C-2 *A layout of a net.*

el. Figure C-4 shows the reduced net model and Figure C-5 shows the lumped capacitance model.

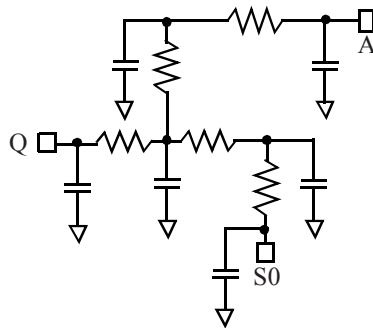


Figure C-3 *Distributed net (D_NET) model.*

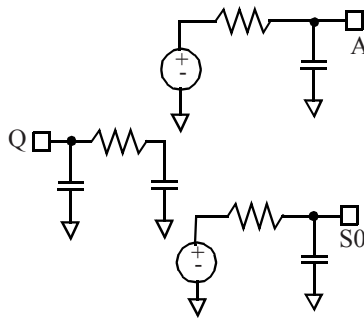


Figure C-4 *Reduced net (R_NET) model.*

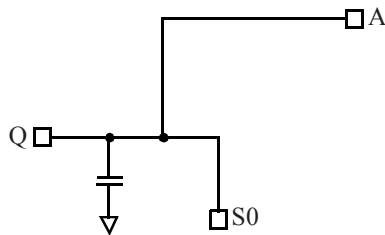


Figure C-5 *Lumped capacitance model.*

Interconnect parasitics depends on process. SPEF supports the specification of best-case, typical, and worst-case values. Such triplets are allowed for R , L and C values, port slews and loads.

By providing a name map consisting of a map of net names and instance names to indices, the SPEF file size is made effectively smaller, and more importantly, all long names appear in only one place.

A SPEF file for a design can be split across multiple files and can also be hierarchical.

C.2 Format

The format of a SPEF file is as follows.

```
header_definition
[ name_map ]
[ power_definition ]
[ external_definition ]
[ define_definition ]
internal_definition
```

The *header definition* contains basic information such as the SPEF version number, design name and units for R , L and C . The *name map* specifies the mapping of net names and instance names to indices. The *power definition* declares the power nets and ground nets. The *external definition* defines the ports of the design. The *define definition* identifies instances, whose SPEF is described in additional files. The *internal definition* contains the guts of the file, which are the parasitics of the design.

Figure C-6 shows an example of a header definition.

```
*SPEF "IEEE 1481-1998"
*DESIGN "ddrphy"
*DATE "Thu Oct 21 00:49:32 2004"
*VENDOR "SGP Design Automation"
*PROGRAM "Galaxy-RCXT"
*VERSION "V2000.06 "
*DESIGN_FLOW "PIN_CAP NONE" "NAME_SCOPE
LOCAL"
*DIVER /
*DELIMITER :
*BUS_DELIMITER [ ]
*T_UNIT 1.00000 NS
*C_UNIT 1.00000 FF
*R_UNIT 1.00000 OHM
*L_UNIT 1.00000 HENRY

// A comment starts with the two characters "//".

// TCAD_GRD_FILE /cad/13lv/galaxy-rcxt/
t013s6ml_fsg.nxtgrd
// TCAD_TIME_STAMP Tue May 14 22:19:36 2002
```

Figure C-6 *A header definition.*

***SPEF** name

specifies the SPEF version.

***DESIGN** name

specifies the design name.

***DATE** string

specifies the time stamp when the file was created.

***VENDOR** string

specifies the vendor tool that was used to create the SPEF.

***PROGRAM** string

specifies the program that was used to generate the SPEF.

***VERSION** string

specifies the version number of the program that was used to create the SPEF.

***DESIGN_FLOW** string string string . . .

specifies at what stage the SPEF file was created. It describes information about the SPEF file that cannot be derived by reading the file. The pre-defined string values are:

- **EXTERNAL_LOADS** : External loads are fully specified in the SPEF file.
- **EXTERNAL_SLEWS** : External slews are fully specified in the SPEF file.
- **FULL_CONNECTIVITY** : Logical netlist connectivity is present in the SPEF.
- **MISSING_NETS** : Some logical nets may be missing from the SPEF file.
- **NETLIST_TYPE_VERILOG** : Uses Verilog HDL type naming conventions.
- **NETLIST_TYPE_VHDL87** : Uses VHDL87 naming convention.
- **NETLIST_TYPE_VHDL93** : Uses VHDL93 netlist naming convention.
- **NETLIST_TYPE_EDIF** : Uses EDIF type naming convention.
- **ROUTING_CONFIDENCE** *positive_integer* : Default routing confidence number for all nets, basically the level of accuracy of the parasitics.

- **ROUTING_CONFIDENCE_ENTRY** *positive_integer* *string* : Supplements the routing confidence values.
- **NAME_SCOPE** **LOCAL** | **FLAT** : Specifies whether paths in the SPEF file are relative to file or to top of design.
- **SLEW_THRESHOLDS** *low_input_threshold_percent*
high_input_threshold_percent : Specifies the default input slew threshold for the design.
- **PIN_CAP** **NONE** | **INPUT_OUTPUT** | **INPUT_ONLY** : Specifies what type of pin capacitances are included as part of total capacitance. The default is **INPUT_OUTPUT**.

The line in the header definition:

***DIVIDER /**

specifies the hierarchy delimiter. Other characters that can be used are *, ;* and */*.

***DELIMITER :**

specifies the delimiter between an instance and its pin. Other possible characters that can be used are *, / ;* or *|*.

***BUS_DELIMITER []**

specifies the prefix and suffix that are used to identify a bit of a bus. Other possible characters that can be used for prefix and suffix are *{ (< , .* and *} ,) > .*

***T_UNIT** *positive_integer* **NS** | **PS**

specifies the time unit.

***C_UNIT** *positive_integer* **PF** | **FF**

specifies the capacitance unit.

```
*R_UNIT positive_integer OHM | KOHM
```

specifies the resistance unit.

```
*L_UNIT positive_integer HENRY | MH | UH
```

specifies the inductance unit.

A *comment* in a SPEF file can appear in two forms.

```
// Comment - until end of line.  
  
/* This comment can  
extend across multiple  
lines */
```

Figure C-7 shows an example of a *name map*. It is of the form:

```
*NAME_MAP  
*positive_integer name  
*positive_integer name  
. . .
```

The name map specifies the mapping of names to unique integer values (their indices). The name map helps in reducing the file size by making all future references of the name by the index. A name can be a net name or an instance name. Given the name map in Figure C-7, the names can later be referenced in the SPEF file by using their index, such as:

```
*364:D           // D pin of instance  
                // mcdll_write_data/write19/d_out_2x_reg_19  
*11172:Y         // Y pin of instance  
                // Tie_VSSQ_assign_buf_318_N_1
```



```

*NAME_MAP
*1 memclk
*2 memclk_2x
*3 reset_
*4 refresh
*5 resync
*6 int_d_out[63]
*7 int_d_out[62]
*8 int_d_out[61]
*9 int_d_out[60]
*10 int_d_out[59]
*11 int_d_out[58]
*12 int_d_out[57]
...
*364 mcdll_write_data/write19/d_out_2x_reg_19
*366 mcdll_write_data/write20/d_out_2x_reg_20
*368 mcdll_write_data/write21/d_out_2x_reg_21
...
*5423 mcdll_read_data/read21/capture_data[53]
...
*5426 mcdll_read_data/read21/capture_pos_0[21]
...
*11172 Tie_VSSQ_assign_buf_318_N_1
...
*14954 test_se_15_S0
*14955 wr_sdly_course_enc[0]_L0
*14956 wr_sdly_course_enc[0]_L0_1
*14957 wr_sdly_course_enc[0]_S0

```

Figure C-7 *A name map.*

```

*5426:116      // Internal node of net
               // mcdll_read_data/read21/capture_pos_0[21]
*5426:10278    // Internal node of net *5426
*12           // The net int_d_out[57]

```

The name map thus avoids repeating long names and their paths by using their unique integer representation.

The *power definition* section defines the power and ground nets.

```
*POWER_NETS net_name net_name . . .
*GROUND_NETS net_name net_name . . .
```

Here are some examples.

```
*POWER_NETS VDDQ
*GROUND_NETS VSSQ
```

The *external definition* contains the definition of the logical and physical ports of the design. Figure C-8 shows an example of logical ports. Logical ports are described in the form:

```
*PORTS
port_name direction { conn_attribute }
port_name direction { conn_attribute }
. . .
```

where a *port_name* can be the port index of form **positive_integer*. The *direction* is **I** for input, **O** for output and **B** for bidirectional. *Connection attributes* are optional, and can be the following:

- ***C** number number : Coordinates of the port.
- ***L** par_value : Capacitive load of the port.
- ***S** par_value par_value : Defines the shape of the waveform on the port.
- ***D** cell_type : Defines the driving cell of the port.

Physical ports in a SPEF file are defined using:

```
*PHYSICAL_PORTS
pport_name direction { conn_attribute }
pport_name direction { conn_attribute }
. . .
```

```
*PORTS
*1 I
*2 I
*3 I
*4 I
*5 I
*6 I
*7 I
*8 I
*9 I
*10 I
*11 I
...
*450 O
*451 O
*452 O
*453 O
*454 O
*455 O
*456 O
```

Figure C-8 *An external definition.*

The *define definition* section defines entity instances that are referenced in the current SPEF file but whose parasitics are described in additional SPEF files.

```
*DEFINE instance_name { instance_name } entity_name
*PDEFINE physical_instance entity_name
```

The ***PDEFINE** is used when the entity instance is a physical partition (instead of a logical hierarchy). Here are some examples.

```
*DEFINE core/u1ddrphy core/u2ddrphy "ddrphy"
```

This implies that there would be another SPEF file with a ***DESIGN** value of `ddrphy` - this file would contain the parasitics for the design `ddrphy`. It is

possible to have physical and logical hierarchy. Any nets that cross the hierarchical boundaries have to be described as distributed nets (`D_NET`).

The *internal definition* forms the guts of the SPEF file - it describes the parasitics for the nets in the design. There are basically two forms: the *distributed net*, `D_NET`, and the *reduced net*, `R_NET`. Figure C-9 shows an example of a distributed net definition.

```
*D_NET *5426 0.899466

*CONN
*I *14212:D I *C 21.7150 79.2300
*I *14214:Q O *C 21.4950 76.6000 *D DFFQX1

*CAP
1 *5426:10278 *5290:8775 0.217446
2 *5426:10278 *16:3754 0.0105401
3 *5426:10278 *5266:9481 0.0278254
4 *5426:10278 *5116:9922 0.113918
5 *5426:10278 0.529736

*RES
1 *5426:10278 *14212:D 0.340000
2 *5426:10278 *5426:10142 0.916273
3 *5426:10142 *14214:Q 0.340000
*END
```

Figure C-9 *Distributed net parasitics for net *5426.*

In the first line,

```
*D_NET *5426 0.899466
```

`*5426` is the net index (see name map for the net name) and `0.899466` is the total capacitance value on the net. The capacitance value is the sum of all capacitances on the net including cross-coupling capacitances that are assumed to be grounded, and including load capacitances. It may or may not

include pin capacitances depending on the setting of `PIN_CAP` in the `*DESIGN_FLOW` definition.

The *connectivity* section describes the drivers and loads for the net. In:

```
*CONN
*I *14212:D I *C 21.7150 79.2300
*I *14214:Q O *C 21.4950 76.6000 *D DFFQX1
```

`*I` refers to an internal pin (`*P` is used for a port), `*14212:D` refers to the `D` pin of instance `*14212` which is an index (see name map for actual name). “`I`” says that it is a load (input pin) on the net. “`O`” says that it is a driver (output pin) on the net. `*C` and `*D` are as defined earlier in connection attributes - `*C` defines the coordinates of the pin and `*D` defines the driving cell of the pin.

The *capacitance* section describes the capacitances of the distributed net. The capacitance unit is as specified earlier with `*C_UNIT`.

```
*CAP
1 *5426:10278 *5290:8775 0.217446
2 *5426:10278 *16:3754 0.0105401
3 *5426:10278 *5266:9481 0.0278254
4 *5426:10278 *5116:9922 0.113918
5 *5426:10278 0.529736
```

The first number is the capacitance identifier. There are two forms of capacitance specification; the first through fourth are of one form and the fifth is of the second form. The first form (first through fourth) specifies the cross-coupling capacitances between two nets, while the second form (with id 5) specifies the capacitance to ground. So in capacitance id 1, the cross-coupling capacitance between nets `*5426` and `*5290` is 0.217446. And in capacitance id 5, the capacitance to ground is 0.529736. Notice that the first node name is necessarily the net name for the `D_NET` that is being described. The positive integer following the net index (10278 in `*5426:10278`) specifies an internal node or junction point. So capacitance id 4 states that there

is a coupling capacitance between net `*5426` with internal node 10278 and net `*5116` with internal node 9922, and the value of this coupling capacitance is 0.113918.

The *resistance section* describes the resistances of the distributed net. The resistance unit is as specified with `*R_UNIT`.

```
*RES
1 *5426:10278 *14212:D 0.340000
2 *5426:10278 *5426:10142 0.916273
3 *5426:10142 *14214:Q 0.340000
```

The first field is the resistance identifier. So there are three resistance components for this net. The first one is between the internal node `*5426:10278` to the `D` pin on `*14212` and the resistance value is 0.34. The capacitance and resistance section can be better understood with the RC network shown pictorially in Figure C-10.

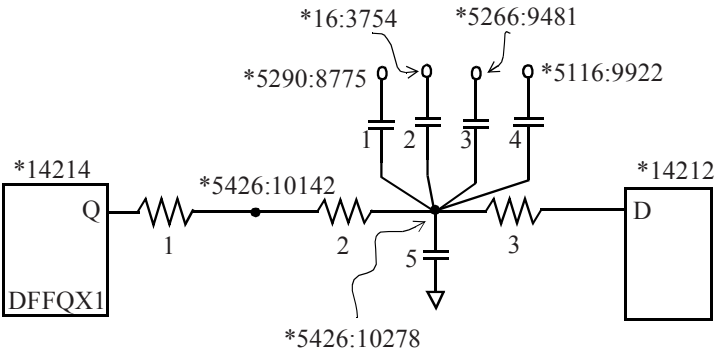


Figure C-10 RC for net `*5426`.

Figure C-11 shows another example of a distributed net. This net has one driver and two loads and the total capacitance on the net is 2.69358. Figure C-12 shows the RC network that corresponds to the distributed net specification.

```

*D_NET *5423 2.69358

*CONN
*I *14207:D I *C 21.7450 94.3150
*I *14205:D I *C 21.7450 90.4900
*I *14211:Q O *C 21.4900 83.8800 *D DFFQX1

*CAP
1 *5423:10107 *547:12722 0.202686
2 *5423:10107 *5116:10594 0.104195
3 *5423:10107 *5233:9552 0.208867
4 *5423:10107 *5265:9483 0.0225810
5 *5423:10107 *267:9668 0.0443454
6 *5423:10107 *5314:7853 0.120589
7 *5423:10212 *2109:996 0.0293744
8 *5423:10212 *5187:7411 0.526945
9 *5423:14640 *6577:10075 0.126929
10 *5423:10213 1.30707

*RES
1 *5423:10107 *5423:10212 2.07195
2 *5423:10107 *5423:10106 0.340000
3 *5423:10212 *5423:10211 0.340000
4 *5423:10212 *5423:14640 1.17257
5 *5423:14640 *5423:10213 0.340000
6 *5423:10213 *14207:D 0.0806953
7 *5423:10211 *14205:D 0.210835
8 *5423:10106 *14211:Q 0.0932139
*END

```

Figure C-11 *Another example of a distributed net *5423.*

In general, an internal definition can comprise of the following specifications:

- **D_NET**: Distributed RC network form of a logical net.
- **R_NET**: Reduced RC network form of a logical net.
- **D_PNET**: Distributed form of a physical net.
- **R_PNET**: Reduced form of a physical net.

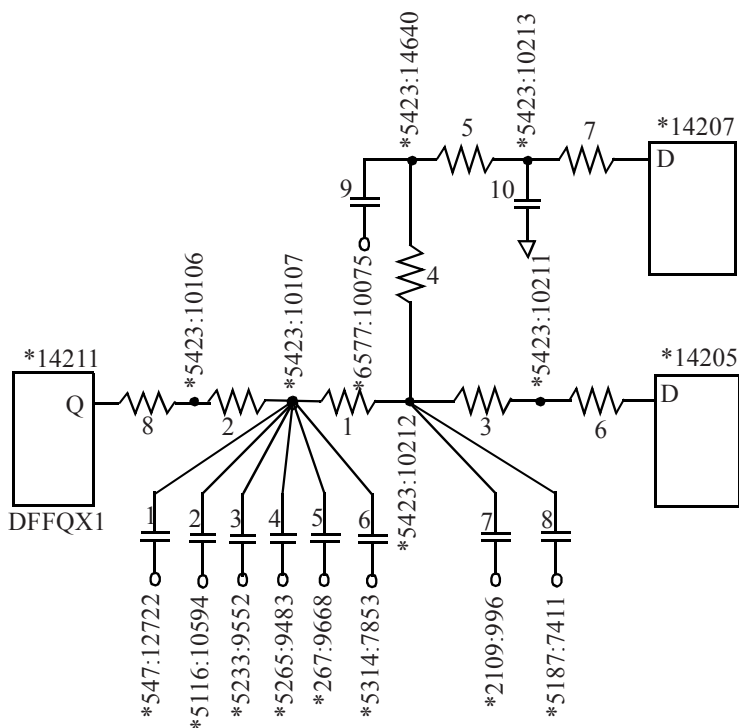


Figure C-12 *RC network for D_NET*5423.*

Here is the syntax.

```
*D_NET net_index total_cap [*V routing_confidence ]
[ conn_section ]
[ cap_section ]
[ res_section ]
[ inductance_section ]
*END

*R_NET net_index total_cap [ *V routing_confidence ]
[ driver_reduction ]
```

***END**

```
*D_PNET pnet_index total_cap [*V routing_confidence ]  
  [ pconn_section ]  
  [ pcap_section ]  
  [ pres_section ]  
  [ pinduc_section ]
```

***END**

```
*R_PNET pnet_index total_cap [*V routing_confidence ]  
  [ pdriver_reduction ]
```

***END**

The *inductance section* is used to specify inductances and the format is similar to the resistance section. The ***v** is used to specify the accuracy of the parasitics of the net. These can be specified individually with a net or can be specified globally using the ***DESIGN_FLOW** statement with the **ROUTING_CONFIDENCE** value, such as:

```
*DESIGN_FLOW "ROUTING_CONFIDENCE 100"
```

which specifies that the parasitics were extracted after final cell placement and final route and 3d extraction was used. Other possible values of routing confidence are:

- 10: Statistical wireload model
- 20: Physical wireload model
- 30: Physical partitions with locations, and no cell placement
- 40: Estimated cell placement with steiner tree based route
- 50: Estimated cell placement with global route
- 60: Final cell placement with steiner route
- 70: Final cell placement with global route
- 80: Final cell placement, final route, 2d extraction
- 90: Final cell placement, final route, 2.5d extraction

- 100: Final cell placement, final route, 3d extraction

A *reduced net* is a net that has been reduced from a distributed net form. There is one driver reduction section for each driver on a net. The driver reduction section is of the form:

```
*DRIVER pin_name
*CELL cell_type
// Driver reduction: one such section for each driver
// of net:
*C2_R1_C1 cap_value res_value cap_value
*LOADS // One following set for each load on net:
*RC pin_name rc_value
*RC pin_name rc_value
. . .
```

The `*c2_r1_c1` shows the parasitics for the pie model on the driver pin of the net. The `rc_value` in the `*RC` construct is the Elmore delay ($R \cdot C$). Figure C-13 shows an example of a reduced net SPEF and Figure C-14 shows the

```
*R_NET *1200 2.995
*DRIVER *1201:Q
*CELL SEDFFX1
*C2_R1_C1 0.511 2.922 0.106
*LOADS
*RC *1202:A 1.135
*RC *1203:A 0.946
*END
```

Figure C-13 *Reduced net example.*

RC network pictorially.

A *lumped capacitance* model is described using either a `*D_NET` or a `*R_NET` construct with just the total capacitance and with no other information. Here are examples of lumped capacitance declarations.

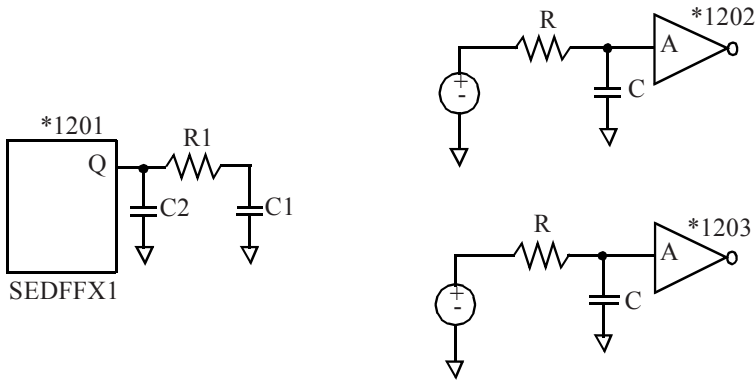


Figure C-14 *Reduced net model.*

```

*D_NET *1 80.2096
*CONN
*I *2:Y O *L 0 *D CLKMX2X2
*P *1 O *L 0
*END

*R_NET *17 58.5204
*END

```

Values in a SPEF file can be in a triplet form that represents the process variations, such as:

0.243:0.269:0.300

0.243 is the best-case value, 0.269 is the typical value and 0.300 is the worst-case value.

C.3 Complete Syntax

This section describes the complete syntax¹ of a SPEF file.

A character can be escaped by preceding with a backslash (\). Comments come in two forms: // starts a comment until end of line, while /* . . . */ is a multi-line comment.

In the following syntax, bold characters such as **l** [are part of the syntax. All constructs are arranged alphabetically and the start symbol is `SPEF_file`.

```
alpha ::= upper | lower

bit_identifier ::=
    identifier
    | <identifier><prefix_bus_delim><digit>{<digit>}
    [ <suffix_bus_delim> ]

bus_delim_def ::=
    *BUS_DELIMITER prefix_bus_delim [ suffix_bus_delim ]

cap_elem ::=
    cap_id node_name par_value
    | cap_id node_name node_name2 par_value

cap_id ::= pos_integer

cap_load ::= *L par_value

cap_scale ::= *C_UNIT pos_number cap_unit

cap_sec ::= *CAP cap_elem { cap_elem }

cap_unit ::= PF | FF

cell_type ::= index | name
```

1. Syntax is reprinted here with permission from IEEE Std. 1481-1999, Copyright 1999, by IEEE. All rights reserved.

```
cnumber ::= ( real_component imaginary_component )

complex_par_value ::=
    cnumber
  | number
  | cnumber:cnumber:cnumber
  | number:number:number

conf ::= pos_integer

conn_attr ::= coordinates | cap_load | slews | driving_cell

conn_def ::=
    *P external_connection direction { conn_attr }
  | *I internal_connection direction { conn_attr }

conn_sec ::=
    *CONN conn_def { conn_def } { internal_node_coord }

coordinates ::= *C number number

date ::= *DATE qstring

decimal ::= [sign]<digit>{<digit>}.{<digit>}

define_def ::= define_entry { define_entry }

define_entry ::=
    *DEFINE inst_name { inst_name } entity
  | *PDEFINE physical_inst entity

design_flow ::= *DESIGN_FLOW qstring [ qstring ]

design_name ::= *DESIGN qstring

digit ::= 0 - 9

direction ::= I | B | O

driver_cell ::= *CELL cell_type

driver_pair ::= *DRIVER pin_name
```

APPENDIX C Standard Parasitic Extraction Format (SPEF)

driver_reduc ::= driver_pair driver_cell pie_model load_desc

driving_cell ::= *D cell_type

d_net ::=
 *D_NET net_ref total_cap [routing_conf]
 [conn_sec]
 [cap_sec]
 [res_sec]
 [induc_sec]
 *END

d_pnet ::=
 *D_PNET pnet_ref total_cap [routing_conf]
 [pconn_sec]
 [pcap_sec]
 [pres_sec]
 [pinduc_sec]
 *END

entity ::= qstring

escaped_char ::= \<escaped_char_set>

escaped_char_set ::= <special_char> | “

exp ::= <radix><exp_char><integer>

exp_char ::= E | e

external_connection ::= port_name | pport_name

external_def ::=
 port_def [physical_port_def]
 | physical_port_def

float ::=
 decimal
 | fraction
 | exp

fraction ::= [sign].<digit>{<digit>}

```
ground_net_def ::= *GROUND_NETS net_name { net_name }
```

```
hchar ::= . | / | : | |
```

```
header_def ::=  
    SPEF_version  
    design_name  
    date  
    vendor  
    program_name  
    program_version  
    design_flow  
    hierarchy_div_def  
    pin_delim_def  
    bus_delim_def  
    unit_def
```

```
hierarchy_div_def ::= *DIVIDER hier_delim
```

```
hier_delim ::= hchar
```

```
identifier ::= <identifier_char>{<identifier_char>}
```

```
identifier_char ::=  
    <escaped_char>  
    | <alpha>  
    | <digit>  
    | _
```

```
imaginary_component ::= number
```

```
index ::= *<pos_integer>
```

```
induc_elem ::= induc_id node_name node_name par_value
```

```
induc_id ::= pos_integer
```

```
induc_scale ::= *L_UNIT pos_number induc_unit
```

```
induc_sec ::= *INDUC induc_elem { induc_elem }
```

```
induc_unit ::= HENRY | MH | UH
```

APPENDIX C Standard Parasitic Extraction Format (SPEF)

```
inst_name ::= index | path

integer ::= [ sign ] <digit> { <digit> }

internal_connection ::= pin_name | pnode_ref

internal_def ::= nets { nets }

internal_node_coord ::= *N internal_node_name coordinates

internal_node_name ::= <net_ref> <pin_delim> <pos_integer>

internal_pnode_coord ::= *N internal_pnode_name coordinates

internal_pnode_name ::= <pnet_ref> <pin_delim> <pos_integer>

load_desc ::= *LOADS rc_desc { rc_desc }

lower ::= a - z

mapped_item ::=
    identifier
  | bit_identifier
  | path
  | name
  | physical_ref

name ::= qstring | identifier

name_map ::= *NAME_MAP name_map_entry { name_map_entry }

name_map_entry ::= index mapped_item

neg_sign ::= -

nets ::= d_net | r_net | d_pnet | r_pnet

net_name ::= net_ref | pnet_ref

net_ref ::= index | path

net_ref2 ::= net_ref
```

```

node_name ::=
    external_connection
  | internal_connection
  | internal_node_name
  | pnode_ref

node_name2 ::=
    node_name
  | <pnet_ref><pin_delim><pos_integer>
  | <net_ref2><pin_delim><pos_integer>

number ::= integer | float

partial_path ::= <hier_delim><bit_identifier>

partial_physical_ref ::= <hier_delim><physical_name>

par_value ::= float | <float>:<float>:<float>

path ::=
    [<hier_delim>]<bit_identifier>{<partial_path>}
    [<hier_delim>]

pcap_elem ::=
    cap_id pnode_name par_value
  | cap_id pnode_name pnode_name2 par_value

pcap_sec ::= *CAP pcap_elem { pcap_elem }

pconn_def ::=
    *P pexternal_connection direction { conn_attr }
  | *I internal_connection direction { conn_attr }

pconn_sec ::=
    *CONN pconn_def { pconn_def } { internal_pnode_coord }

pdriver_pair ::= *DRIVER internal_connection

pdriver_reduc ::= pdriver_pair driver_cell pie_model load_desc

pexternal_connection ::= pport_name

physical_inst ::= index | physical_ref

```

```
physical_name ::= name

physical_port_def ::=
  *PHYSICAL_PORTS pport_entry { pport_entry }

physical_ref ::= <physical_name>{<partial_physical_ref>}

pie_model ::=
  *C2_R1_C1 par_value par_value par_value

pin ::= index | bit_identififer

pinduc_elem ::= induc_id pnode_name pnode_name par_value

pinduc_sec ::=
  *INDUC
  pinduc_elem
  { pinduc_elem }

pin_delim ::= hchar

pin_delim_def ::= *DELIMITER pin_delim

pin_name ::= <inst_name><pin_delim><pin>

pnet_ref ::= index | physical_ref

pnet_ref2 ::= pnet_ref

pnode ::= index | name

pnode_name ::=
  pexternal_connection
  | internal_connection
  | internal_pnode_name
  | pnode_ref

pnode_name2 ::=
  pnode_name
  | <net_ref><pin_delim><pos_integer>
  | <pnet_ref2><pin_delim><pos_integer>

pnode_ref ::= <physical_inst><pin_delim><pnode>
```

```
pole ::= complex_par_value

pole_desc ::= *Q pos_integer pole { pole }

pole_residue_desc ::= pole_desc residue_desc

port_def ::=
    *PORTS
    port_entry
    { port_entry }

pos_decimal ::= <digit>{<digit>}.{<digit>}

port ::= index | bit_identifier

port_entry ::= port_name direction { conn_attr }

port_name ::= [<inst_name><pin_delim>]<port>

pos_exp ::= pos_radix exp_char integer

pos_float ::= pos_decimal | pos_fraction | pos_exp

pos_fraction ::= .<digit>{<digit>}

pos_integer ::= <digit>{<digit>}

pos_number ::= pos_integer | pos_float

pos_radix ::= pos_integer | pos_decimal | pos_fraction

pos_sign ::= +

power_def ::=
    power_net_def [ ground_net_def ]
    | ground_net_def

power_net_def ::= *POWER_NETS net_name { net_name }

pport ::= index | name

pport_entry ::= pport_name direction { conn_attr }
```

APPENDIX C Standard Parasitic Extraction Format (SPEF)

```
pport_name ::= [<physical_inst><pin_delim>]<pport>

prefix_bus_delim ::= { | [ | ( | < | : | .

pres_elem ::= res_id pnode_name pnode_name par_value

pres_sec ::=
    *RES
    pres_elem
    { pres_elem }

program_name ::= *PROGRAM qstring

program_version ::= *VERSION qstring

qstring ::= "{qstring_char}"

qstring_char ::= special_char | alpha | digit | white_space | _

radix ::= decimal | fraction

rc_desc ::= *RC pin_name par_value [ pole_residue_desc ]

real_component ::= number

residue ::= complex_par_value

residue_desc := *K pos_integer residue { residue }

res_elem ::= res_id node_name node_name par_value

res_id ::= pos_integer

res_scale ::= *R_UNIT pos_number res_unit

res_sec ::=
    *RES
    res_elem
    { res_elem }

res_unit ::= OHM | KOHM

routing_conf ::= *V conf
```

```

r_net ::=
    *R_NET net_ref total_cap [ routing_conf ]
    { driver_reduc }
    *END

r_pnet ::=
    *R_PNET pnet_ref total_cap [ routing_conf ]
    { pdriver_reduc }
    *END

sign ::= pos_sign | neg_sign

slews ::= *S par_value par_value [ threshold threshold ]

special_char ::=
    ! | # | $ | % | & | ` | ( | ) | * | + | , | - | . | / | : | ; | < | = | >
    | ? | @ | [ | \ | ] | ^ | ' | { | | | } | ~

SPEF_file ::=
    header_def
    [ name_map ]
    [ power_def ]
    [ external_def ]
    [ define_def ]
    internal_def

SPEF_version ::= *SPEF qstring

suffix_bus_delim ::= ] | } | ) | >

threshold ::=
    pos_fraction
    | <pos_fraction>:<pos_fraction>:<pos_fraction>

time_scale ::= *T_UNIT pos_number time_unit

time_unit ::= NS | PS

total_cap ::= par_value

unit_def ::= time_scale cap_scale res_scale induc_scale

upper ::= A - Z

```

APPENDIX C Standard Parasitic Extraction Format (SPEF)

```
vendor ::= *VENDOR qstring  
white_space ::= space | tab
```

□

Bibliography

1. [ARN51] Arnoldi, W.E., *The principle of minimized iteration in the solution of the matrix eigenvalue problem*, Quarterly of Applied Mathematics, Volume 9, pages 17–25, 1951.
2. [BES07] Best, Roland E., *Phase Locked Loops: Design, Simulation and Applications*, McGraw-Hill Professional, 2007.
3. [BHA99] Bhasker, J., *A VHDL Primer, 3rd edition*, Prentice Hall, 1999.
4. [BHA05] Bhasker, J., *A Verilog HDL Primer, 3rd edition*, Star Galaxy Publishing, 2005.
5. [CEL02] Celik, M., Larry Pileggi and Altan Odabasioglu, *IC Interconnect Analysis*, Springer, 2002.
6. [DAL08] Dally, William J., and John Poulton, *Digital Systems Engineering*, Cambridge University Press, 2008.
7. [ELG05] Elgamel, Mohamed A. and Magdy A. Bayoumi, *Interconnect Noise Optimization in Nanometer Technologies*, Springer, 2005.

8. [KAN03] Kang, S.M. and Yusuf Leblebici, *CMOS Digital Integrated Circuits Analysis and Design, 3rd Edition*, New York: McGraw Hill, 2003.
9. [LIB] *Liberty Users Guide*, available at "<http://www.opensourceliberty.org>".
10. [MON51] Monroe, M.E., *Theory of Probability*, New York: McGraw Hill, 1951.
11. [MUK86] Mukherjee, A., *Introduction to nMOS & CMOS VLSI Systems Design*, Prentice Hall, 1986.
12. [NAG75] Nagel, Laurence W., *SPICE2: A computer program to simulate semiconductor circuits*, Memorandum No. ERL-M520, University of California, Berkeley, May 1975.
13. [QIA94] Qian, J., S. Pullela and L. Pillegi, *Modeling the "Effective Capacitance" for the RC Interconnect of CMOS Gates*, IEEE Transaction on CAD of ICS, Vol 13, No 12, Dec 94.
14. [RUB83] Rubenstein, J., P. Penfield, Jr., and M. A. Horowitz, *Signal delay in RC tree networks*, IEEE Trans. Computer-Aided Design, Vol. CAD-2, pp. 202-211, 1983.
15. [SDC07] *Using the Synopsys Design Constraints Format: Application Note*, Version 1.7, Synopsys Inc., March 2007.
16. [SRI05] Srivastava, A., D. Sylvester, D. Blaauw, *Statistical Analysis and Optimization for VLSI: Timing and Power*, Springer, 2005.

□

Index

12-value delay 481
1-value delay 481
2.5d extraction 547
2d extraction 547
2-value delay 481
3d extraction 548
6-value delay 481

A

absolute path delay 474
absolute port delay 475
AC noise rejection 156
AC specifications 318
AC threshold 159
accurate RC 7
active clock edge 277
active edge 61, 236
active power 88, 412
additional margin 32
additional pessimism 32
aggressor net 165, 167
aggressors 149
all_clocks 449
all_inputs 449
all_outputs 449
all_registers 449
annotator 496
approximate RC 8
area specification 94

area units 100
async default path group 279
asynchronous control 277
asynchronous design 5
asynchronous input arc 74
asynchronous inputs 60

B

backannotation 467, 496
backslash 550
backward-annotation 485
balanced tree 108
BCF 41, 420
best-case fast 41, 227, 370, 420
best-case process 534
best-case tree 108
best-case value 549
bidirectional skew timing check 483
black box 73
byte lane 121

C

C value 534
CAC 336, 341
capacitance identifier 543
capacitance section 543
capacitance specification 543
capacitance unit 100, 538, 543